

Notebook Build Audit & Execution System

Unified Framework Specification

Construction, Audit, and Resource Analysis

“Codificar experiencia humana, validar con supervisión sistemática.”

— Encoding human expertise, validating through systematic oversight

Compiled from: NotebookBuildAudit.tex & InitialProposalDev.tex

July 27, 2026

Contents

I	Codificar Experiencia Humana	2
1	The Challenge	2
2	Notebook Construction Framework	2
3	LLM-as-a-Judge as the Evaluation Paradigm	4
3.1	Motivation	4
3.2	Deterministic Evaluation vs. LLM Judging	5
3.3	Prompt Engineering as Judge Design	6
3.4	Known Limitations	6
3.5	Position within This Work	7
4	Prompt Crafting — Encoding Expert Knowledge	8
5	Audit Framework — Human-in-the-Loop Validation	10
II	Architecture	12
6	Conceptual Architecture	12
7	Logical Architecture	12
8	Physical Architecture	14
III	Computational Costs	14
9	Infrastructure Costs	15
10	LLM Inference Costs	15
11	Development Investment	16
IV	Energetic Costs — Proposal for a Third Pillar	17
12	Motivation	17
13	Energy Sources in the System	17
14	Proposal: Resource Efficiency as a Third Pillar	18
15	Implementation Path for Pass 7	19

Part I

Codificar Experiencia Humana

The fundamental thesis of this work is that rigorous notebook engineering requires *encoding human expertise* into repeatable, verifiable structures — and then *validating* those structures through a systematic, human-in-the-loop audit protocol. This part presents the two complementary frameworks that realise this vision.

1 The Challenge

Computational notebooks (`.ipynb`) have become the dominant medium for data science and machine learning workflows. However, they introduce systemic risks that compromise reproducibility, correctness, and auditability at every stage of the lifecycle:

- **Untrusted notebook execution** — notebooks often originate from external collaborators, open-source repositories, or AI-assisted code generation without any deterministic guarantee that they run safely or correctly.
- **Lack of deterministic builds** — ad hoc environment pinning, missing dependency declarations, and absent seed controls cause notebooks to produce different results across machines or sessions, undermining scientific validity.
- **No structured audit protocol** — teams lack a systematic, multi-pass methodology for reviewing notebooks before production deployment, publication, or merge. Existing code review practices are insufficient for the unique failure modes of notebook-based ML pipelines: data leakage through preprocessing ordering, cross-validation contamination, inseparable training/inference logic.

The **Notebook Build Audit & Execution System** addresses these gaps through two tightly coupled frameworks: a forward-looking *Construction Framework* that guides notebook authors in producing well-structured artifacts, and a backward-looking *Audit Framework* that systematically diagnoses weaknesses in existing notebooks.

2 Notebook Construction Framework

The Construction Framework is a *forward-looking prescription*: it guides the notebook author in producing a well-structured, reproducible, and maintainable artifact from the ground up. It is organized into three sequential phases.

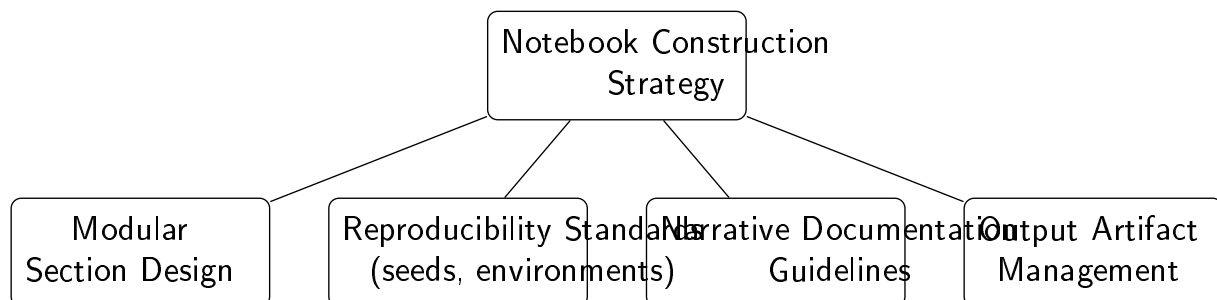


Figure 1: High-level pillars of the Notebook Construction Framework.

Phase 1 — Scaffold

Establish the structural and reproducibility foundation before writing any logic:

- Define the notebook's **single responsibility** — one notebook should correspond to one coherent workflow or experiment.
- Create standard section headers as markdown cells, following this canonical order:
 1. Environment & Dependencies
 2. Configuration & Global Parameters
 3. Data Ingestion
 4. Preprocessing & Feature Engineering
 5. Model Definition & Training
 6. Evaluation & Metrics
 7. Artifact Export (models, plots, reports)
 8. Conclusions & Next Steps
- Pin the execution environment upfront via `requirements.txt`, `conda.yaml`, or `pyproject.toml`.
- Set global reproducibility controls: random seeds, deterministic flags, and device configuration.

Phase 2 — Write

Author each section incrementally, following these discipline rules:

- Work through **one section at a time** (Divide and Conquer principle).
- Accompany every code cell with a markdown explanation of intent and expected output.
- Avoid burying logic inside loops or helper calls without commentary.
- Prefer **explicit variable passing** over hidden or global state to maintain cell independence.
- Summarise repetitive code patterns with a general description and highlight meaningful variations; three or more similar blocks should be treated as a refactoring candidate.
- Route all persisted outputs (models, plots, reports) through a single, versioned export convention defined at the start of the section — never write artifacts ad hoc from arbitrary cells.

Phase 3 — Validate During Writing

Continuously verify correctness as each section is completed:

- Restart the kernel and run all cells after completing each section.
- Confirm **cell idempotency**: every cell should produce the same result when re-executed independently.
- Verify that outputs match expectations before proceeding to the next section.
- Ensure cell execution order is strictly linear and free of hidden dependencies.

- Confirm that exported artifacts land in the designated output location with the expected naming and versioning convention before moving to the next section.

The Construction Framework encodes human expertise about *how to build rigorous notebooks* into a repeatable process. The expertise — section discipline, reproducibility hygiene, incremental validation — is the “codified experience” that the author brings to each new notebook.

3 LLM-as-a-Judge as the Evaluation Paradigm

The proposed notebook auditing framework follows the emerging **LLM-as-a-Judge (LAJ)** paradigm [1], in which a powerful language model evaluates the outputs of another model (or of itself under controlled prompting) instead of relying solely on human reviewers. This approach is particularly suitable for notebook audits because many evaluation criteria — such as code clarity, logical coherence, documentation quality, or methodological soundness — cannot be measured using deterministic metrics alone.

Unlike traditional automatic evaluation metrics (e.g., BLEU or ROUGE), which require reference outputs and perform poorly on open-ended reasoning tasks, an LLM judge can evaluate responses using qualitative criteria similar to those employed by expert reviewers. Recent work has shown that sufficiently capable language models achieve agreement with human evaluators comparable to inter-human agreement when judging open-ended responses [1].

3.1 Motivation

Notebook auditing combines objective and subjective evaluation tasks.

Objective tasks include:

- dependency validation,
- execution reproducibility,
- train/test split verification,
- artifact existence,
- environment completeness.

These can often be evaluated deterministically through programmatic checks.

Conversely, **subjective tasks** include:

- quality of documentation,
- conceptual coherence,
- methodological soundness,
- explanation quality,
- maintainability,
- deployment readiness.

These require semantic reasoning and therefore benefit from an LLM acting as an evaluator.

The proposed framework intentionally separates these two categories rather than attempting to solve both with a single mechanism.

3.2 Deterministic Evaluation vs. LLM Judging

The audit framework adopts a layered evaluation strategy.

Layer 1 — Deterministic Verification

Whenever correctness can be established objectively, deterministic evaluation is preferred. Examples include:

- successful notebook execution,
- dependency resolution,
- version pinning,
- artifact generation,
- file existence,
- reproducibility checks.

These evaluations are binary and reproducible.

Layer 2 — LLM-as-a-Judge

Only after deterministic checks have been completed does the framework invoke an LLM judge to assess properties requiring semantic interpretation. Examples include:

- structural organisation,
- readability,
- methodological consistency,
- code explanation quality,
- deployment recommendations.

This mirrors the evaluation philosophy adopted by recent agent benchmarks, where objective verification forms the first layer of validation while model-based evaluation is reserved for inherently subjective tasks.

Budget-Aware Allocation (ROBIN-HOOD)

A critical practical consideration arises because LLM judgments are stochastic — querying the same pair multiple times yields different scores due to sampling randomness. Score variance is highly heterogeneous: a factual check like “dependency resolution” may have variance near zero, while a subjective assessment like “methodological soundness” can vary significantly. Given a fixed query budget B across K evaluation pairs, optimal allocation is non-trivial [5].

Recent work formalises this as a multi-armed bandit problem: each pair is an arm with unknown mean s_i and variance σ_i^2 , and the goal is to minimise worst-case estimation error $\max_i |s_i - \hat{s}_i|$ under budget B . The **ROBIN-HOOD** algorithm [5] dynamically allocates queries based on estimated score variances using upper confidence bounds, concentrating resources where uncertainty is highest. It achieves a worst-case error of $\tilde{O}(\sqrt{\sum_i \sigma_i^2 / B})$ with near-optimal allocation, and experiments on **HelpSteer2** show it matches uniform allocation accuracy at **half the budget**.

This is directly relevant to the notebook audit framework: the six-pass protocol contains both low-variance passes (Structural Overview, Reproducibility Check) and high-variance passes (ML

Correctness Audit, Deployment Readiness). A production system should therefore weight its LLM query budget toward passes where the evaluation is most uncertain, rather than querying each pass uniformly.

3.3 Prompt Engineering as Judge Design

An LLM judge is only as reliable as its evaluation protocol [2].

Following the G-Eval methodology, the prompts presented in this work do not ask the model to provide a single holistic score. Instead, each prompt decomposes notebook evaluation into explicit review criteria.

Rather than asking

“Is this notebook good?”

the evaluator is instructed to inspect predefined dimensions such as:

- reproducibility,
- data integrity,
- methodological correctness,
- implementation quality,
- deployment readiness.

The resulting structured reports resemble formal peer-review checklists instead of free-form opinions, improving consistency across repeated evaluations.

This principle directly motivates the progressive prompt hierarchy introduced in Section 4, where increasingly constrained prompts guide the model from coarse inspection toward structured multi-pass auditing.

3.4 Known Limitations

Although LLM judges exhibit strong agreement with human evaluators, they remain susceptible to systematic biases [3, 4]:

- **Position bias** — preference for the first candidate presented.
- **Verbosity bias** — longer responses often receive higher scores despite similar quality.
- **Self-enhancement bias** — models tend to rate their own generations more favourably than those produced by competing models.
- **Reasoning limitations** — evaluation quality decreases when the judge must solve difficult mathematical or logical problems before assessing correctness.

These limitations motivate the design choices adopted throughout the audit framework:

- decomposition into multiple audit passes,
- explicit evaluation criteria,
- preference for deterministic verification whenever possible,
- avoidance of single holistic quality scores.

Rather than treating the LLM as an oracle, the framework views it as a structured reviewer operating within clearly defined evaluation boundaries.

The Generation–Evaluation Dissociation

A deeper structural limitation, recently formalised by the *Metonym Game* [6], is that **generating and evaluating are dissociated skills**. In a peer-community benchmark where LLMs both create content and rate each other’s creations, singular value decomposition of the ratings matrix reveals that:

- The strongest generators are **middling judges** — producing high-quality content does not imply the ability to evaluate it reliably.
- The sharpest judges are **mid-pack generators** — evaluation skill is scarcer and partially independent of generative capability.
- Factual rating competence correlates with GPQA Diamond at $r = 0.92$, validating that the spectral decomposition recovers genuine evaluation ability.

This finding has direct implications for notebook auditing: using the same LLM model to generate audit reports (Passes 1–6) and to evaluate their own correctness may conflate two distinct capabilities. A more robust design, following the “jury” proposal [7], would employ a **panel of diverse models** — some optimised for structured reasoning (Passes 3–4), others for stylistic judgment (Passes 5–6) — and aggregate their assessments to mitigate individual model biases.

3.5 Position within This Work

The notebook auditing framework proposed in this document should therefore be interpreted as a specialised instance of the LLM-as-a-Judge paradigm.

Its primary contribution is not a new evaluation model, but a structured auditing protocol that combines:

1. deterministic verification wherever objective correctness exists;
2. prompt-engineered LLM judging for semantic evaluation;
3. incremental multi-pass analysis that progressively refines the assessment while reducing cognitive overload for both the evaluator and the reader.

This hybrid strategy combines the reproducibility of automated software testing with the flexibility of modern language-model evaluation, providing a practical methodology for AI-assisted review of computational notebooks.

Extensions Informed by Recent Advances

Two lines of recent work directly extend the framework’s practical reach:

Budget-aware query allocation. The ROBIN-HOOD algorithm [5] addresses a critical operational question: given a fixed LLM query budget, how should it be distributed across the six passes to minimise worst-case estimation error? Because pass variance differs (low for Reproducibility Check, high for ML Correctness Audit), a variance-adaptive allocation significantly outperforms uniform querying. This is the natural next step for a production deployment.

Panel-based evaluation (jury). The Metonym Game [6] demonstrates that generating and evaluating are dissociated skills — the best generators are not the best judges. Combined with

the jury proposal [7], this motivates replacing a single LLM judge with a **council of diverse models**, each specialising in passes aligned with its strengths (e.g., a smaller model for structural checks, a larger one for ML correctness). The architecture’s existing **LLMProvider** strategy pattern already supports this extension without structural changes.

4 Prompt Crafting — Encoding Expert Knowledge

To operationalise the Construction and Audit frameworks through an LLM, human expertise must be encoded into structured prompts that guide the model’s reasoning. This section presents a three-level prompt engineering framework that maps cleanly onto the six-pass audit protocol.

The Three-Level Structure

1. **Level 1 — Conceptual** (Passes 1–2): Structural Overview + Reproducibility Check. Establishes whether the notebook is coherent and re-runnable at all, before any deep inspection is worth doing.
2. **Level 2 — Methodological** (Passes 3–4): Data Integrity Review + ML Correctness Audit. Addresses whether the pipeline and modelling logic are scientifically sound — the “does this work correctly?” layer.
3. **Level 3 — Implementation** (Passes 5–6): Code Quality Review + Deployment Readiness. Addresses *how* the working logic is written and packaged — the most granular, code-level layer.

Prompt Iterations (Lap Progression)

The prompt templates themselves evolve through successive refinements:

Lap 1 — Coarse Evaluation: A broad, unstructured task statement that asks the LLM to analyse the notebook holistically. It requests a phase 1 documentation (purpose, code block descriptions, pseudocode for complex functions) and a phase 2 critical audit (cross-block consistency, redundancy, contradictory logic, error handling). Yields an initial gate decision.

Lap 2 — Refined Evaluation: Introduces structural rules for block numbering (visual order, not execution order), consolidation of imports into a single “Environment Setup” block, filtering of markdown and error cells, and explicit output format requirements. Adds a five-point extraction template: Logic Flow, Data Transformations, Model/Algorithm Operations, Key Variables/Parameters, Side Effects.

Lap 3 — Structured Evaluation: Formalises the three-level hierarchy (Conceptual, Methodological, Implementation) as a deterministic mapping onto the six-pass framework. Each level has its own deliverable requirements, gate decisions, and scoring rubric.

Level 1 — Conceptual Purpose (Detail)

This level determines whether the notebook is coherent and reliably re-executable before investing time in deeper review.

Procedure:

1. *Scope the audit* — Record user-specified focus areas or exclusions.
2. *Map the structure* — Assess section headers, confirm single coherent purpose, verify linear execution order.

3. *Scan for immediate red flags* — Missing outputs, broken imports, orphaned cells.
4. *Check reproducibility inputs* — Dependency pinning, global seed configuration.
5. *Check reproducibility risks* — Hardcoded paths, credentials, environment-specific assumptions.

Deliverables: Section map, preliminary red flags list, recorded focus-area scope, reproducibility risk score (Low / Moderate / High).

Gate decision: If the notebook fails end-to-end execution or lacks a coherent single purpose, decide whether to proceed to Level 2 or flag Level 1 as blocking.

Level 2 — Methodological Purpose (Detail)

This level evaluates the scientific soundness of the data pipeline and ML methodology.

Procedure:

1. *Audit the data pipeline* — Train/test split ordering, data leakage detection, missing-data consistency, type/shape validation at ingestion.
2. *Audit ML correctness* — Metric appropriateness, cross-validation integrity, hyperparameter tuning boundaries, baseline comparison.
3. *Cross-check across pipeline stages* — Verify that transformations applied during preprocessing are correctly reflected in downstream modelling and evaluation.

Deliverables: Data pipeline integrity report, ML correctness checklist.

Gate decision: If methodological flaws are found (leakage, invalid cross-validation, metric misuse), decide whether to halt or proceed to Level 3 with flagged caveats.

Level 3 — Implementation Purpose (Detail)

This level assesses code maintainability and production suitability.

Procedure:

1. *Code quality review* — Identify repetitive blocks (three-block threshold), dead code, naming quality, output hygiene.
2. *Deployment readiness check* — Versioned artifact export, inference/training separability, resource documentation, environment completeness, data privacy.

Deliverables: Code quality and smell report, deployment readiness score (Low / Moderate / High).

Weighted Evaluation Dimensions

For external audits (e.g., markdown exports), a weighted multi-dimensional scoring system is used:

- **Reproducibility & Self-Containment (40%)** — Highest priority. Dependency declarations, output expectations, placeholders.
- **Narrative & Structure (35%)** — Section transitions, heading hierarchy, audience identification, reading flow.
- **Purpose & Depth (25%)** — Explicit primary purpose, learning path (tutorials), hypothesis–method–results separation (experiments), interpretable outputs.

This weighted system ensures that critical reproducibility and structural flaws receive proportionate attention before stylistic or depth-based concerns.

5 Audit Framework — Human-in-the-Loop Validation

The Audit Framework is a *backward-looking diagnostic*: it guides a reviewer, collaborator, or AI assistant in systematically assessing the quality, correctness, and risk profile of an existing notebook.

Guiding Tactics

Before executing the passes, the reviewer employs these tactics:

- Divide the notebook into logical modules, prioritising critical ML pipeline blocks over repetitive exploration cells.
- Use hierarchical descriptions: high-level summaries first, drill-downs later.
- Summarise repetitive code patterns rather than analysing each individually.
- When a human collaborator specifies focus areas in advance, scope the six-pass protocol to those areas rather than executing all passes uniformly.

The Six-Pass Audit Protocol

Where a pass deliverable is a score, the reviewer rates on a three-level scale — *Low Risk*, *Moderate Risk*, *High Risk* — based on the number and severity of unresolved items identified within that pass.

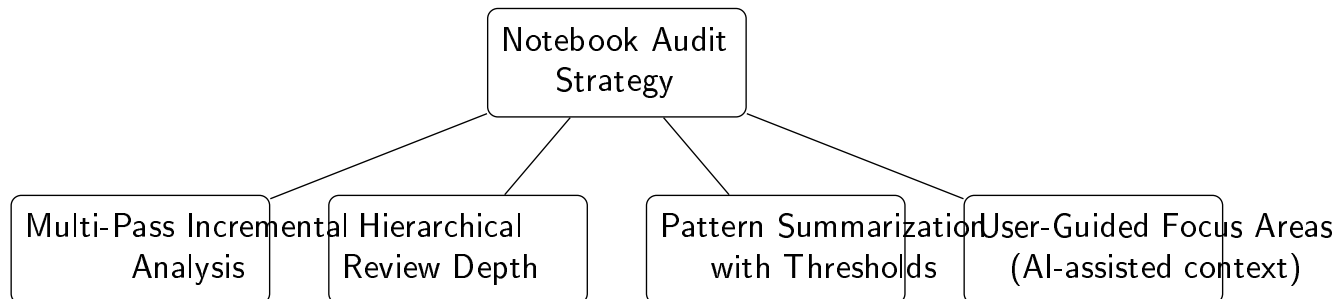


Figure 2: High-level pillars of the Notebook Audit Framework.

Pass 1 — Structural Overview: Establish a high-level map of the notebook before examining any logic:

- Record any user-specified focus areas or exclusions.
- Assess clear, navigable section headers and single coherent purpose.
- Verify linear, safe cell execution order.
- Identify immediately visible red flags (missing outputs, broken imports, orphaned cells).

Deliverable: Section map, preliminary red flags list, recorded focus-area scope.

Pass 2 — Reproducibility Check: Assess whether the notebook can be reliably re-executed:

- Verify dependency pinning (`requirements.txt`, `conda.yaml`).

- Confirm global, consistent random seed configuration.
- Flag hardcoded file paths, credentials, environment assumptions.
- Determine end-to-end runnability on a fresh kernel.

Deliverable: Reproducibility risk score (Low / Moderate / High).

Pass 3 — Data Integrity Review: Examine the data pipeline for correctness and leakage risks:

- Confirm train/test split is performed **before** any preprocessing.
- Identify data leakage (e.g., scaling fitted on full dataset).
- Verify consistent missing-data handling across splits.
- Check data type, shape, and distribution validation at ingestion.

Deliverable: Data pipeline integrity report.

Pass 4 — ML Correctness Audit: Evaluate the soundness of the machine learning methodology:

- Confirm metric appropriateness for task and class distribution.
- Verify correct cross-validation application without leakage.
- Ensure hyperparameter tuning uses validation data only.
- Check performance contextualisation against a meaningful baseline.

Deliverable: ML correctness checklist.

Pass 5 — Code Quality Review: Assess maintainability and clarity of the code:

- Flag repetitive blocks exceeding the three-block threshold.
- Identify dead code, unused imports, redundant assignments.
- Assess meaningful and consistent variable/function naming.
- Check for bloated cell outputs.

Deliverable: Code quality and smell report.

Pass 6 — Deployment Readiness: Determine production or publication suitability:

- Verify versioned model artifact export.
- Confirm separable inference and training logic.
- Check documented compute and memory constraints.
- Ensure complete, up-to-date environment export.
- Flag data privacy or PII handling concerns.

Deliverable: Deployment readiness score (Low / Moderate / High).

Applying the Two Frameworks

The appropriate framework depends on the context of engagement with the notebook:

Use the Construction Framework when: starting a new notebook or experiment; restructuring a disorganised notebook from scratch; or creating a reusable notebook template.

Use the Audit Framework when: reviewing a collaborator’s notebook; preparing for production deployment or academic publication; conducting a pre-merge or pre-deployment review.

Use Both Frameworks iteratively when: authoring and self-reviewing during active development; collaborating or pair-programming in real time; or refactoring an existing notebook while preserving its structure.

The two frameworks form a feedback loop:

Construction Phase → produces notebook artifact → Audit Phase → diagnoses weaknesses → recommendations feed back into revised Construction → ...

This closure applies to notebooks that begin in the Building Phase. For notebooks entering via the audit-only path, the loop remains open unless the notebook is subsequently revised.

Part II

Architecture

This part presents the system architecture at three levels of abstraction: conceptual (the mental model), logical (the component model), and physical (the deployment model).

6 Conceptual Architecture

The conceptual architecture is defined by the relationship between the two frameworks and the three-level audit model that governs LLM-driven analysis.

Core Concepts

- **Notebook Lifecycle** — A notebook exists in one of two states: under construction (authoring) or under audit (review). The feedback loop connects both states.
- **Three-Level Audit Model** — The six-pass protocol is grouped into three levels (Conceptual, Methodological, Implementation) that correspond to increasing depth of analysis. Gate decisions between levels allow early termination when blocking issues are found.
- **Human-in-the-Loop** — The system is diagnostic, not prescriptive. Every gate decision and pass deliverable is presented to a human reviewer who makes the final call. The LLM provides structured evidence; the human provides judgment.
- **Provider Agnosticism** — The LLM orchestration layer abstracts over local (Ollama, llama.cpp) and cloud (OpenAI) providers through a strategy pattern, allowing the same audit logic to run in air-gapped or connected environments.

Framework Feedback Loop

7 Logical Architecture

The logical architecture decomposes the system into its core components and their responsibilities.

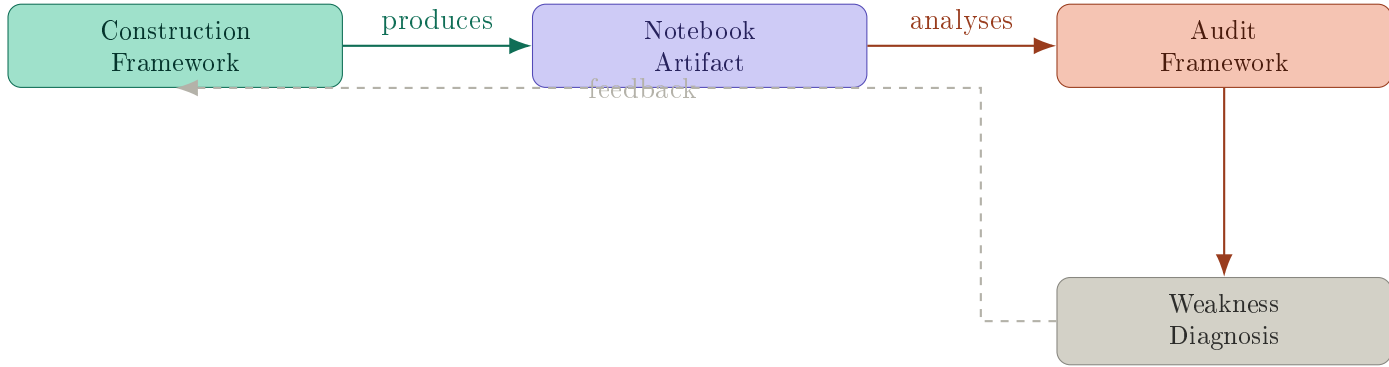


Figure 3: Conceptual feedback loop between the Construction and Audit Frameworks.

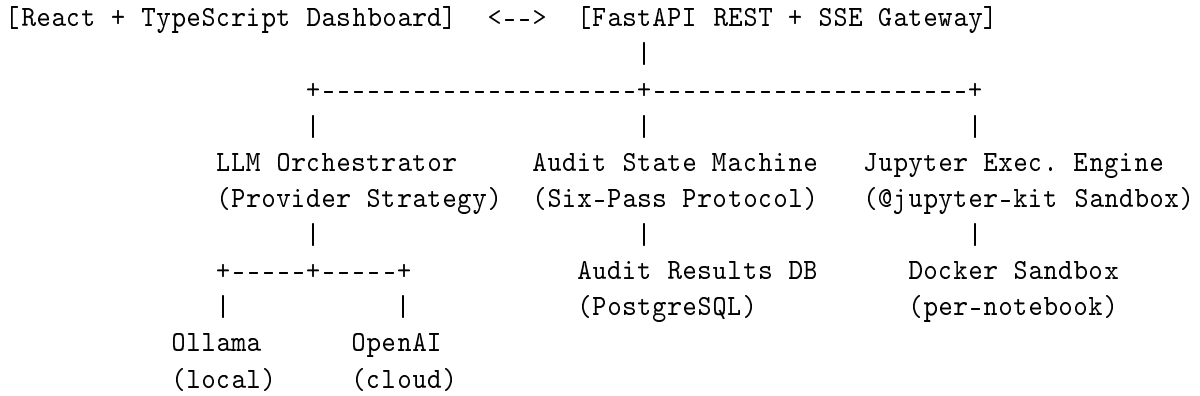


Figure 4: Logical component diagram of the Notebook Build Audit & Execution System.

System Components

Component Responsibilities

React + TypeScript Dashboard: Single-page application for audit session management, notebook upload, provider routing configuration, pass scope selection, and live audit progress visualisation. Communicates with the backend via REST and Server-Sent Events (SSE).

FastAPI REST + SSE Gateway: Async-first Python backend exposing a REST API for audit session CRUD, notebook upload, and configuration. SSE endpoints stream per-pass audit deliverables in real time, enabling live progress without polling.

LLM Orchestrator (Provider Strategy): Implements the `LLMProvider` abstract base class with concrete implementations for Ollama, `llama.cpp`, and OpenAI. The `LLMOrchestrator` service selects the appropriate provider at runtime based on audit configuration (privacy, latency, capability requirements). Prompt templates from the three-level framework are parameterised and injected per audit pass. The strategy pattern is designed to accept additional providers (Anthropic, OpenCode) post-launch without architectural changes.

Audit State Machine: Each notebook audit is modelled as a state machine transitioning through `Scope` $\rightarrow$ `Pass1` $\rightarrow$ `Pass2` $\rightarrow$ $\dots$ $\rightarrow$ `Pass6`. Gate decisions between Levels 1, 2, and 3 are configurable checkpoints — the system can halt at a blocking issue or continue to surface further findings.

Jupyter Execution Engine (@jupyter-kit): Programmatic Jupyter kernel management, cell-level execution, output capture, and metadata inspection within a sandboxed Docker container.

PostgreSQL + SQLAlchemy 2.0 (async): Relational storage for audit sessions, notebook metadata, audit results, and user configuration. Schema migrations managed via Alembic.

Key Architectural Decisions

Strategy Pattern for LLM Providers: The `LLMProvider` abstract base class defines `generate(prompt, **kwargs)`. Concrete implementations handle provider-specific API formatting. Runtime provider selection is based on audit configuration — local providers for offline/privacy-sensitive audits, cloud providers for high-capability inference.

Audit State Machine: State transitions with gate checkpoints. Each gate can be configured to halt on failure or continue with caveats, giving the human reviewer full control over audit depth.

Server-Sent Events for Real-Time Logs: The FastAPI backend streams per-pass deliverables to the frontend via SSE, enabling live progress without polling overhead.

8 Physical Architecture

Deployment Topology

The system is deployed via Docker Compose with the following services:

Table 1: Physical deployment services.

Service	Image/Base	Role
Frontend	Node 20 + Vite build	Serves the React SPA
Backend	Python 3.11+ / FastAPI	REST + SSE API
Database	PostgreSQL 15	Persistent storage
Sandbox	Python 3.11+ (per-notebook)	Isolated <code>@jupyter-kit</code> execution
LLM Local	Ollama / <code>llama.cpp</code>	On-device inference

Resource Isolation

Each notebook audit spawns a short-lived Docker container with the notebook’s declared dependencies pre-installed. The container runs a headless Jupyter kernel managed by `@jupyter-kit` and is destroyed after the audit completes, providing full isolation between audit sessions.

Provider Deployment Options

- **Local (air-gapped)**: Ollama or `llama.cpp` running on the same host or local network. Zero external API calls. Suitable for privacy-sensitive or regulated environments.
- **Cloud (high-capability)**: OpenAI API (and potentially Anthropic, OpenCode) for maximum model capability. Requires internet connectivity and API key configuration.

Part III

Computational Costs

This part analyses the computational resources required to operate the system, covering infrastructure, LLM inference, and development investment.

9 Infrastructure Costs

Hardware Requirements

The base deployment requires a server with the following minimum specifications:

Table 2: Minimum hardware requirements.

Component	Specification
CPU	8+ cores (x86_64 or ARM64)
RAM	32 GB minimum; 64 GB recommended for local LLM inference
GPU	Optional; NVIDIA with 8+ GB VRAM for local GPU-accelerated inference
Storage	100 GB SSD (system + models + databases)
Network	100 Mbps (cloud provider mode requires internet access)

Deployment Cost Estimate

Table 3: Estimated infrastructure costs (self-hosted, one-time).

Item	USD	COP (approx.)
Server PC / Workstation	\$1,000	≈ 3,900,000
Initial cloud hosting (3 months)	\$300	≈ 1,170,000
Total (first quarter)	\$1,300	≈ 5,070,000

10 LLM Inference Costs

Inference costs depend on the chosen provider and the number of audit passes executed:

Table 4: LLM inference cost comparison.

Provider	Cost Model	Notes
Ollama (local)	\$0 (hardware only)	No per-token cost. Requires sufficient local RAM/VRAM.
<code>llama.cpp</code> (local)	\$0 (hardware only)	CPU-optimised inference; no GPU required.
OpenAI API	Per-token pricing	Variable; depends on model (GPT-4o, GPT-4o-mini) and prompt length.

Local inference (`llama.cpp` or Ollama) is recommended for privacy-sensitive or high-volume audit environments, while cloud API inference is suitable for capability-demanding passes where connectivity is available.

11 Development Investment

Team and Timeline

The system is estimated to require **8 weeks (2 months)** with a team of 2 developers:

Table 5: Project phases and duration.

Phase	Description	Duration
Phase 1	Environment & Dockerization. Monorepo structure, Docker Compose, per-notebook sandbox, dependency pinning, environment export.	2 weeks
Phase 2	FastAPI Backend & LLM Integration. REST API, <code>LLMProvider</code> strategy pattern, <code>AuditStateMachine</code> , <code>@jupyter-kit</code> integration, SSE streaming.	3 weeks
Phase 3	Frontend Interface. React dashboard, audit session management, live progress view, risk score visualisation, provider configuration.	2 weeks
Phase 4	Integration & QA. End-to-end testing, prompt template validation, security review, documentation.	1 week

Total Development Cost

Table 6: Total development cost breakdown.

Cost Item	USD	COP (approx.)
Developer Costs (2 devs $\times$ 2 months $\times$ \$1,000/mo)	\$4,000	$\approx$ 15,600,000
Hardware / Server PC	\$1,000	$\approx$ 3,900,000
Total Project Cost	\$5,000	$\approx$ 19,500,000

Part IV

Energetic Costs — Proposal for a Third Pillar

This part examines the energetic (energy consumption) dimension of notebook execution and LLM inference, and proposes its consideration as a potential third pillar alongside the Construction and Audit frameworks.

12 Motivation

As LLM-based systems become integral to data science workflows, the energy cost of both notebook execution and LLM inference is no longer negligible. Three converging trends motivate its inclusion as a first-class concern:

1. **LLM inference is energy-intensive.** Running a single large language model query consumes orders of magnitude more energy than traditional computation. A single LLM inference pass can consume 0.01–0.1 kWh depending on model size and hardware [8]. Over hundreds or thousands of audit passes, this accumulates significantly.
2. **Notebook execution is often wasteful.** Unoptimised data pipelines re-run expensive transformations, GPU memory is left allocated between cells, and exploratory dead ends are not cleaned up. The Construction Framework’s discipline rules (cell independence, explicit variable passing) already address some of this indirectly, but energy is not an explicit optimisation target.
3. **Environmental accountability is becoming a requirement.** Academic institutions, funding bodies, and corporate ESG (Environmental, Social, Governance) frameworks increasingly require energy reporting for computational research. An audit system that cannot measure or report energy impact is incomplete for these stakeholders.

13 Energy Sources in the System

The system’s energy consumption comes from three primary sources:

1. Notebook Execution (Sandbox)

Each audit session spawns a Docker container that executes the notebook from scratch. Energy is consumed by:

- CPU cycles for data loading, preprocessing, and modelling code.
- GPU cycles (if available) for model training and inference within the notebook.
- Memory and I/O operations during execution.
- Container orchestration overhead.

2. LLM Inference (Audit Passes)

Each audit pass involves one or more LLM calls. Energy consumption scales with:

- **Model size:** Larger models (70B+ parameters) consume significantly more energy per token than smaller models (7B–13B).

- **Context length:** Longer notebook content means longer prompts, which increases both latency and total energy per pass.
- **Provider efficiency:** Local inference on dedicated hardware may be more or less efficient than cloud-based inference depending on utilisation, hardware generation, and data centre energy mix.

3. Infrastructure Overhead

- Server idle power draw between audit sessions.
- Network transfer for cloud API calls and model downloads.
- Storage I/O for result persistence and model caching.

14 Proposal: Resource Efficiency as a Third Pillar

The central question is: *should computational and energetic costs be elevated to a third pillar, co-equal with the Construction and Audit frameworks?*

Arguments for Inclusion

Completeness: A framework that prescribes how to build and audit notebooks but ignores the resource cost of both activities is incomplete. The three pillars would become: **(1) Construction, (2) Audit, (3) Resource Efficiency.**

Feedback integration: The Construction Framework’s discipline rules could be extended with energy-aware guidance: “prefer lazy loading for large datasets”, “clean up GPU memory between cells”, “profile energy cost of heavy transformations”. The Audit Framework could include an optional Energy Pass measuring notebook energy footprint and suggesting optimisations.

Stakeholder alignment: ESG reporting, grant compliance, and institutional policy increasingly demand energy transparency. A system that can report “this notebook consumed X kWh across N audit passes” provides immediate value to these stakeholders.

Market differentiation: No existing notebook audit tool explicitly measures or optimises for energy consumption. Adding this as a core pillar would distinguish the system in the market.

Arguments against Inclusion

Scope creep: The system is primarily an audit and construction tool for notebook correctness and reproducibility. Adding energy as a co-equal pillar changes the scope from “quality assurance” to “quality assurance + environmental impact”.

Measurement complexity: Accurate per-notebook energy measurement requires specialised tooling (e.g., Intel RAPL, NVIDIA NVML, kernel-level profiling) that may not be available in all deployment environments. Inconsistent measurement would undermine the pillar’s reliability.

Pillar asymmetry: The Construction and Audit frameworks are *methodological* — they prescribe how humans and LLMs should work. Resource Efficiency is *quantitative* — it measures and optimises resource usage. Mixing methodological and quantitative pillars may create conceptual friction.

User burden: Requiring energy reporting and optimisation may deter adoption, especially for teams that simply want to check notebook correctness without environmental accounting.

Proposed Compromise: Optional Energy Module

Rather than elevating Resource Efficiency to a co-equal pillar, we propose its inclusion as an **optional module** within the Audit Framework:

- **Pass 7 — Energy Impact Assessment (optional):** An additional audit pass that measures the notebook’s energy footprint across execution and LLM inference phases. The deliverable is an energy impact report with a three-level rating (Low / Moderate / High).
- **Energy-aware construction guidelines:** The Construction Framework’s Phase 2 (Write) and Phase 3 (Validate) could include optional energy-awareness best practices, gated behind a “green mode” toggle.
- **Measurement layer:** Energy profiling would be implemented as a pluggable backend — RAPL for Intel CPUs, NVML for NVIDIA GPUs, or software estimation models when hardware counters are unavailable. The module would degrade gracefully: “energy data unavailable” when no profiler is present, rather than failing the audit.

Decision Framework

Table 7: Comparison of inclusion strategies for the energy dimension.

Criterion	Full Third Pillar	Optional Module (Pass 7)
Scope	Co-equal with Constr. & Audit	Subordinate to Audit Framework
Measurement	Required for all audits	Optional, graceful degradation
User impact	Mandatory energy reporting	Opt-in, no default burden
Implementation	New framework, processes, docs	Single additional pass + config
Market signal	Strong differentiation	Incremental improvement

Recommendation: Proceed with the **Optional Module approach** (Pass 7) in the initial implementation. Revisit the Third Pillar decision after 6–12 months of production usage, based on user adoption of the energy pass and stakeholder demand for environmental reporting.

15 Implementation Path for Pass 7

If adopted, Pass 7 (Energy Impact Assessment) would be implemented as follows:

Measurement Backends

Table 8: Energy measurement backends by platform.

Platform	Tool	Scope
Linux (Intel)	<code>perf</code> / RAPL	CPU package + DRAM energy
Linux (AMD)	<code>rapl-read</code> / Zenpower	CPU package energy
Linux (NVIDIA GPU)	NVML (<code>pynvml</code>)	GPU energy + memory
macOS (Apple Silicon)	<code>powermetrics</code>	CPU + GPU + ANE
Cloud / fallback	Software estimation model	Platform-independent estimate

Pass Deliverables

Notebook Execution Energy: Total energy consumed during kernel-restart-and-run-all, broken down by section when possible.

LLM Inference Energy: Estimated energy per audit pass, based on model size, context length, and inference duration.

Energy Rating: Low / Moderate / High, contextualised against a baseline (e.g., “2× the energy of a typical notebook of this length”).

Optimisation Suggestions: When energy rating is Moderate or High, suggest specific actions (e.g., “consider using a smaller LLM model for Pass 5”, “this data transformation could be cached instead of recomputed”).

References

- [1] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [2] Y. Liu *et al.*, “G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment,” *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 2511–2522, 2023.
- [3] P. Wang *et al.*, “A Systematic Study of Bias in LLM-as-a-Judge,” *arXiv preprint arXiv:2405.07632*, 2024.
- [4] X. Zhang *et al.*, “Evaluating and Mitigating Positional Bias in LLM-as-a-Judge,” *arXiv preprint arXiv:2404.01234*, 2024.
- [5] A. Saha, A. Wagde, and B. Kveton, “LLM-as-Judge on a Budget,” *arXiv preprint arXiv:2602.15481*, 2026.
- [6] D. Nordfors, “The Metonym Game: A Self-Contained, Self-Consistent LLM Peer-Community Benchmark for Structural Intelligence,” *arXiv preprint arXiv:2606.21008*, 2026.
- [7] P. Verga *et al.*, “Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models,” *arXiv preprint arXiv:2404.18796*, 2024.
- [8] C. Paterson *et al.*, “EcoAssistant: Using LLM Assistant More Affordably and Environmentally Friendly,” *arXiv preprint arXiv:2402.02156*, 2024.

- [9] P. Henderson *et al.*, “Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning,” *Journal of Machine Learning Research*, vol. 21, pp. 1–43, 2020.
- [10] E. Strubell, A. Ganesh, and A. McCallum, “Energy and Policy Considerations for Deep Learning in NLP,” *Proceedings of ACL*, pp. 3645–3650, 2019.
- [11] A. Lacoste, A. Luber, Y. Bengio, and M. Lafrance-Lavoie, “Quantifying the Carbon Emissions of Machine Learning,” *Workshop on Tackling Climate Change with Machine Learning at NeurIPS*, 2019.
- [12] J. Dodge *et al.*, “Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping,” *arXiv preprint arXiv:2002.06305*, 2020.